
'''

////////////////

Software Requirements Specification

for

SafeVault

Version 1.0 approved

Prepared by Group 11

**Felicia Faustine Hidayat
Florencia Olga
Michelle Angela Pang**

BINUS University

25 February 2026

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Intended Audience and Reading Suggestions	1
1.4 Product Scope	1
1.5 References	1
2. Overall Description	2
2.1 Product Perspective	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints	2
2.6 User Documentation	2
2.7 Assumptions and Dependencies	3
3. External Interface Requirements	3
3.1 User Interfaces	3
3.2 Hardware Interfaces	3
3.3 Software Interfaces	3
3.4 Communications Interfaces	3
4. System Features	4
4.1 System Feature 1	4
4.2 System Feature 2 (and so on)	4
5. Other Nonfunctional Requirements	4
5.1 Performance Requirements	4
5.2 Safety Requirements	5
5.3 Security Requirements	5
5.4 Software Quality Attributes	5
5.5 Business Rules	5
6. Other Requirements	5
Appendix A: Glossary	5
Appendix B: Analysis Models	5
Appendix C: To Be Determined List	6

Revision History

Name	Date	Reason For Changes	Version
Group 12	25/02/2026	Initial Project	1.0
Group 11	04/03/2026	Introduction	1.1

Group 11	11/03/2026	Overall description	1.2
Group 11	20/04/2026	External Features Requirements	1.3

1. Introduction

1.1 Purpose

Dokumen Software Requirements Specification (SRS) ini memberikan detail terkait mengenai website SafeVault versi 1.0, yaitu website penyimpanan lokal untuk password yang aman. Tujuan dari SafeVault sendiri adalah menyediakan user dengan cara yang aman dan dapat diandalkan untuk mereka menyimpan, mengelola, dan mengambil kembali kredensial login mereka, termasuk nama website, username, dan password. Sistem SafeVault menggunakan mekanisme autentikasi master password, hashing, dan enkripsi untuk melindungi informasi sensitif dari akses yang tidak terpercaya.

1.2 Document Conventions

Dokumen SRS ini disusun mengikuti struktur dan format standar IEEE untuk spesifikasi kebutuhan perangkat lunak. Penomoran bab dan sub-bab digunakan untuk memudahkan pembacaan dan navigasi antar bagian dalam dokumen.

Setiap pernyataan kebutuhan sistem dituliskan secara jelas. Prioritas kebutuhan tidak diwariskan dari kebutuhan yang lebih tinggi, setiap requirement memiliki prioritasnya masing masing. Tidak terdapat font, warna, atau penanda khusus yang memiliki makna tertentu selain untuk penulisan dan penekanan umum

1.3 Intended Audience and Reading Suggestions

Dokumen Software Requirements Specification (SRS) ini ditujukan kepada:

Pihak Terlibat	Peran
Dosen mata kuliah Software Engineering LM01: Sonya Rapinta Manalu, S.Kom., M.T.I.	Evaluator dan pemberi arahan dalam proses pengembangan sistem
Pengembang Sistem: Kelompok 11	Merancang dan mengembangkan sistem website SafeVault
Pemilik Proyek: Kelompok 12	Memilih ide proyek SafeVault dan memantau perkembangan sistem website SafeVault

1.4 Product Scope

SafeVault merupakan sebuah website yang berfungsi sebagai solusi pengelolaan kredensial pengguna secara aman. Sistem ini dirancang untuk mengurangi resiko kehilangan dan penyalahgunaan password pengguna akibat penyimpanan yang tidak terorganisir dan tidak aman.

Tujuan pengembangan SafeVault adalah menyediakan sarana pengelolaan password yang praktis, dapat diandalkan, dan meningkatkan kesadaran pengguna akan pentingnya perlindungan data pribadi.

Manfaat yang diharapkan dari website ini adalah kemudahan dalam mengelola banyak akun, peningkatan keamanan informasi, dan pengurangan ketergantungan pada layanan penyimpanan password pihak ketiga. SafeVault berfokus pada keamanan, privasi, dan kemandirian pengguna dalam pengelolaan data.

Pada versi 1.0, ruang lingkup SafeVault fokus pada fungsi inti pengelolaan dan penyimpanan password secara lokal dan aman. Pengembangan fitur lanjutan akan dipertimbangkan pada tahap pengembangan berikutnya

Fitur-fitur yang disediakan:

1. Autentikasi master password

Saat pertama kali menggunakan web, pengguna akan membuat master password dan login menggunakan master password. Master password tersebut kemudian akan disimpan dalam bentuk hash. Akses untuk login akan ditolak jika password salah.

2. Penyimpanan kredensial akun

Pengguna dapat menyimpan informasi-informasi akun, seperti nama website atau aplikasi, email atau username, dan password.

3. Enkripsi password

Password akan dienkripsi sebelum disimpan dalam website dan akan didekripsi saat ditampilkan kepada pengguna.

4. Manajemen kredensial

Di fitur ini, pengguna dapat menambahkan akun baru, menghapus akun, melihat daftar akun yang sudah disimpan, dan mengedit data akun.

5. Dashboard web

Menampilkan daftar akun yang sudah tersimpan di dalam web.

1.5 References

1. Garima Chandel, Anu Saji John, and Ashish Sharma, "Web Based Password Management System For Robust Applications", IEEE International Students' Conference on Electrical, Electronics, and Computer Science (SCEECS), 2024.
2. Fernando, W.P.L., Dissanyake, D.A.N.P., Dushmantha, S.G.V.D., Liyanage, D.L.C.P., & Karunatilake, C. "Challenges and Opportunities in Password Management: A review of Current Solutions.", Sri Lanka Journal of Social Sciences and Humanities, 3(2), 9-20, 2023.
3. Hussein Abdulkhaleq Saleh, "Password Managers: A Critical Review of Security, Usability, and Innovative Designs", Journal Computer Sciences Institute, 36, 328-335, 2025.

2. Overall Description

2.1 Product Perspective

SafeVault merupakan sebuah sistem manajemen kata sandi bersifat mandiri (self-contained product) yang dirancang untuk mengelola kredensial dengan mudah dan fleksibel. Produk ini memiliki fungsi bagaikan brankas digital yang memungkinkan pengguna untuk menyimpan data kredensial dalam suatu perangkat dan dapat mengaksesnya di perangkat lain secara real-time.

Dalam persaingan di dunia teknologi, produk ini memposisikan diri dalam perspektif:

- Sinkronisasi ekosistem yang fleksibel: dibandingkan dengan iOS dan Android Passkey yang kuat namun mengunci pengguna dalam sisi ekosistem, SafeVault memberikan fitur sinkronisasi yang

dapat dilakukan antar perangkat Android, macOS, iOS, dan Windows dengan mudah dan tidak terbatas oleh akun vendor.

- Metode autentikasi: SafeVault menggunakan kombinasi antara Master Password dan enkripsi simetris, menyediakan pilihan bagi pengguna yang lebih menyukai untuk melakukan kontrol secara manual dibandingkan menggunakan hardware biometrik tertentu.
- Transparansi dalam penyimpanan: SafeVault menekankan sistem yang berbasis pada Client Side Encryption, dimana data password akan dienkripsi pada perangkat lokal terlebih dahulu, memastikan penyedia layanan tidak dapat melihat data sensitif sekalipun.

2.2 Product Functions

- Sinkronisasi antar perangkat: menyediakan sistem yang dapat memberikan akses data pada perangkat berbeda secara real-time
- Brankas terenkripsi: penyimpanan data sensitif (website, username, password) dengan teknik enkripsi tingkat tinggi (AES-256).
- Autentikasi master: Menggunakan satu sandi kuat utama yang ditentukan pengguna sebagai akses untuk menggunakan aplikasi.
- Manajemen kredensial (CRUD): kemampuan untuk pengguna dalam menambah, mengedit, menghapus, dan mengelompokan data akun.
- Generator kata sandi: fitur tambahan yang memiliki fungsi untuk membuat kata sandi secara acak dan kuat bagi pengguna.

2.3 User Classes and Characteristics

1. Pengguna Umum (End-User)

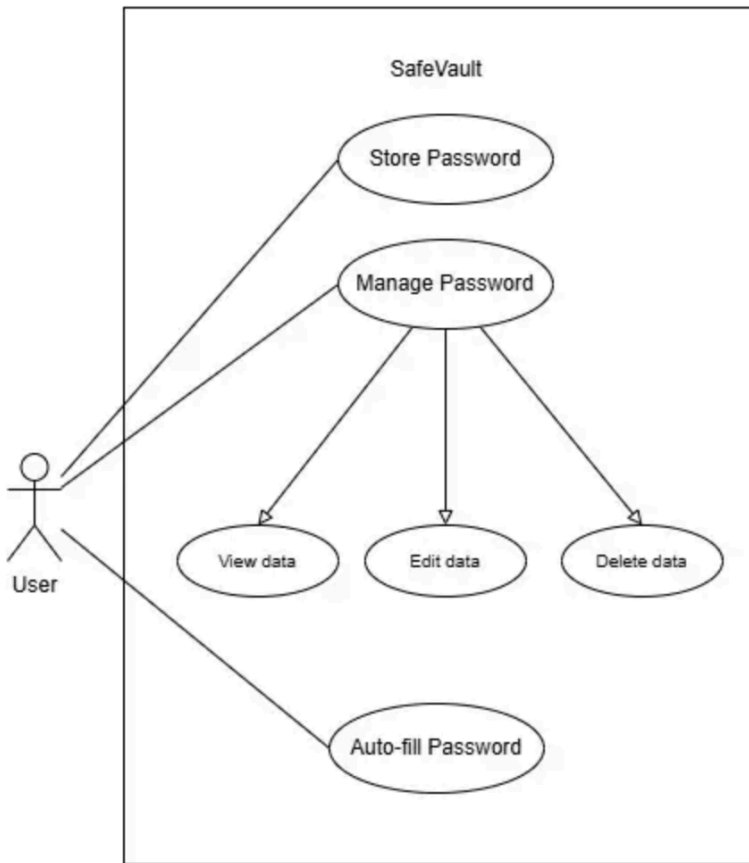
- Karakteristik: Individu dengan mobilitas tinggi yang menggunakan smartphone, laptop, dan *device* yang lainnya.
- Tujuan: Menyimpan, mengelola, dan menyinkronkan password secara aman lintas perangkat.
- Tingkat Teknis: Bervariasi (dari pemula hingga mahir). Mereka membutuhkan *interface* yang intuitif dan proses sinkronisasi yang otomatis tanpa perlu memahami mekanisme enkripsi di belakangnya.

2. Administrator Sistem (SysAdmin)

- Karakteristik: Tenaga teknis yang bertanggung jawab atas infrastruktur cloud SafeVault.
- Tujuan: Memantau kesehatan server, mengelola lalu lintas data (traffic), melakukan backup database cloud, dan memastikan layanan tetap tersedia (high availability) 24/7.
- Tingkat Teknis: Mahir. Memiliki akses ke panel kontrol server tetapi tidak memiliki akses ke konten brankas pengguna karena data terenkripsi secara end-to-end.

Berikut UML dari SafeVault:

2.3.1 Use Case Diagram



2.3.2 Use Case Description

UC-01: Store Password

Use Case ID	UC-01
Use Case Name	Store Password
Actor(s)	User
Description	Allows the user to save a new password entry into the SafeVault system with other data such as username and the site name.
Pre-conditions	The user is logged into SafeVault.
Post-conditions	The new password is successfully stored in the vault.
Main Flow	1. The user selects the option to store a new password. 2. The system prompts the user to enter the password and its related data (site name, username). 3. The user enters the password details and confirms them. 4. The system validates and processes the password. 5. The system shows the password's strength score and asks for confirmation. 6. The system saves password entry. 7. The system confirms successful password storage to the user.
Alternate Flow	• If the password entry is not complete or invalid, the system displays an error and prompts the user to re-enter. • If the password strength score is low, the system will display a warning message. • If the user cancels to save the password, the system will prompt the user to re-enter.
Priority	High

UC-02: Manage Password

Use Case ID	UC-02
Use Case Name	Manage Password
Actor(s)	User
Description	Serves as the central hub use case for managing existing password entries. It extends to View Data, Edit Data, and Delete Data, allowing the user to perform full CRUD operations on their stored passwords.
Pre-conditions	The user is logged into SafeVault and at least one password entry exists in the vault.
Post-conditions	The user's chosen management action (view, edit, or delete) is done successfully.
Main Flow	1. The user navigates to the password management section. 2. The system displays the list of stored password entries. 3. The user selects a password entry. 4. The user chooses an action: View, Edit, or Delete. 5. The system executes the selected sub-use case (UC-03, UC-04, or UC-05).
Inheriting	UC-03 (View Data), UC-04 (Edit Data), UC-05 (Delete Data)
Priority	High

UC-03: View Data

Use Case ID	UC-03
Use Case Name	View Data
Actor(s)	User
Description	Allows the user to view the details of a stored password entry. The password may be shown in masked or revealed form depending on the user's action.
Pre-conditions	The user has selected a password entry from Manage Password (UC-02) and choose option "view".
Post-conditions	The password details are shown to the user.
Main Flow	1. The system ask user for authentication and validates it. 2. The system retrieves and decrypts the selected password entry. 3. The system displays the details (site name, username, password masked by default). 4. The user may toggle to reveal the password.
Alternate Flow	• If the authentication is not valid, the system will ask the user to input again until it's valid or the user clicks cancel. • The user cancels authentication, the system returns to Manage Password (UC-02) without changes.
Inherited by	UC-02 (Manage Password)
Priority	High

UC-04: Edit Data

Use Case ID	UC-04
Use Case Name	Edit Data
Actor(s)	User
Description	Allows the user to modify an existing password entry (such as updating the password, site name, or username, and saves the updated entry back into the vault).
Pre-conditions	The user has selected a password entry from Manage Password (UC-02) and choose option “edit”.
Post-conditions	The password entry is updated and saved.
Main Flow	1. The system ask user for authentication and validates it. 2. The system decrypts the password and displays the current password entry details in editable form. 3. The user modifies the desired fields. 4. The user confirms the changes. 5. The system validates and show the password strength score. 6. The system show confirmation button and processes to updated entry (encrypt the password). 7. The system saves the updated entry. 8. The system confirms the update to the user.
Alternate Flow	• If the authentication is not valid, the system will ask the user to input again until it's valid or the user clicks cancel. • The user cancels authentication, the system returns to Manage Password (UC-02) without changes. • If the update form is not edited, the system displays an error and prompts the update form again. • If the password entry is invalid, the system displays an error and prompts the user to re-enter. • If the password strength score is low, the system will display a warning message. • If the user cancels to save the password, the system will prompt the user to re-enter the update form.
Inherited by	UC-02 (Manage Password)
Priority	High

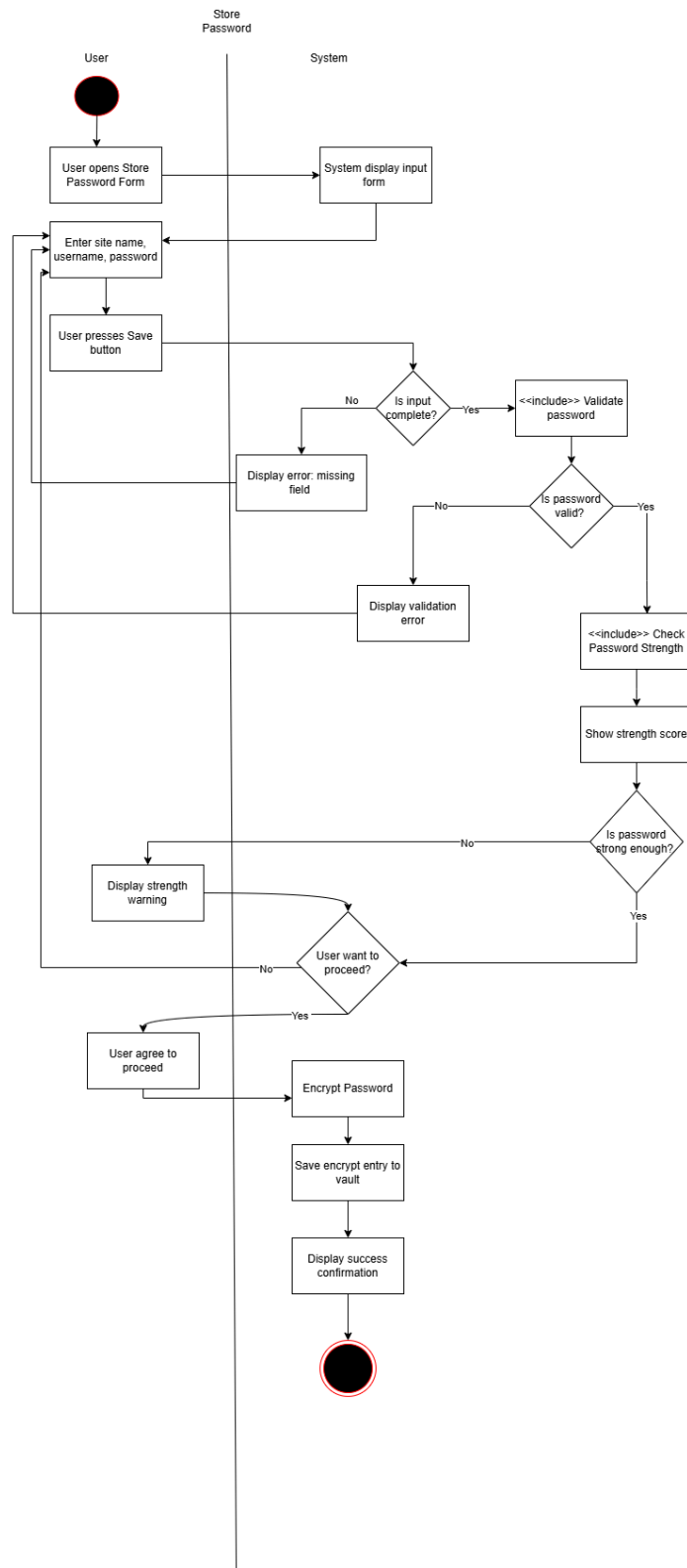
UC-05: Delete Data

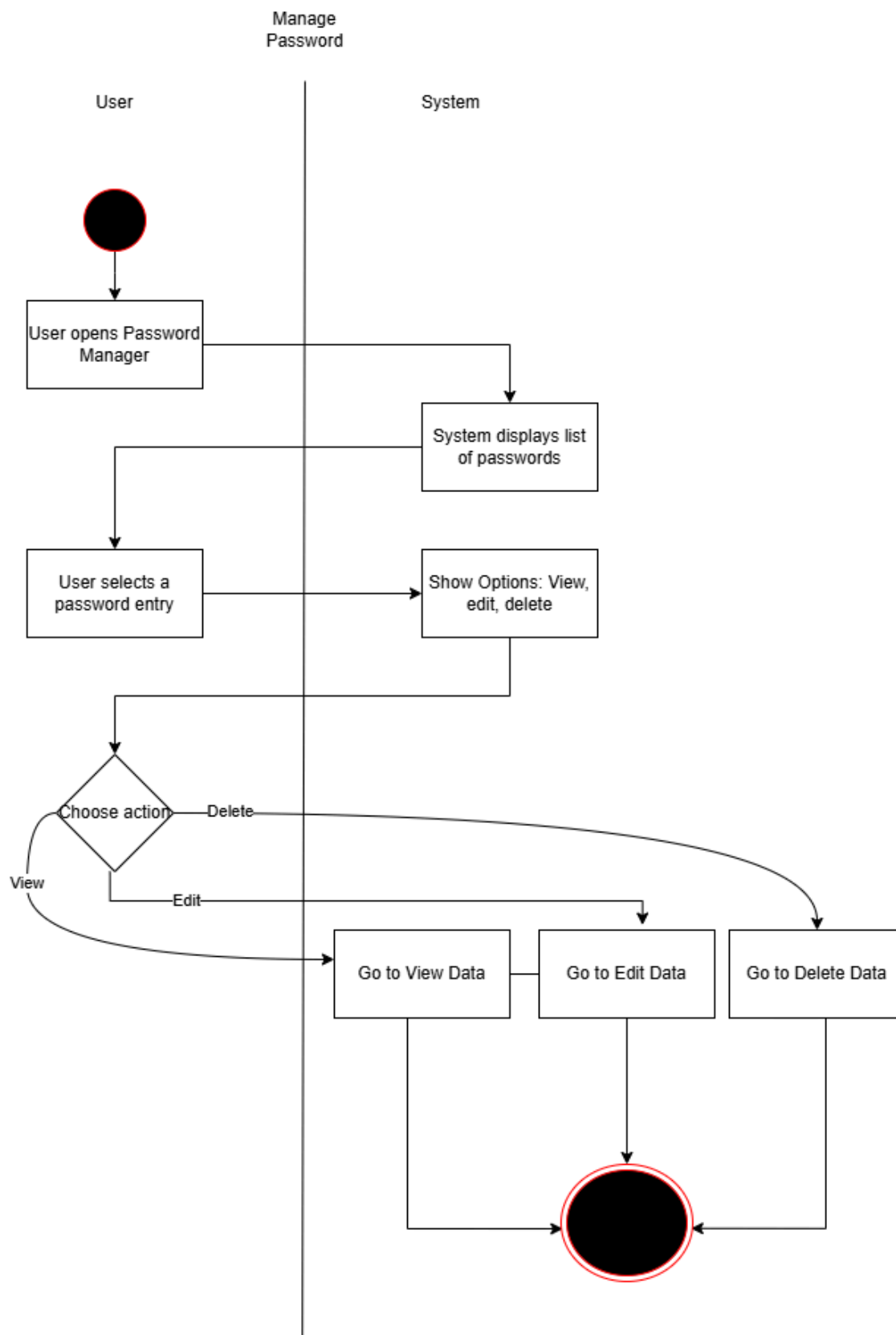
Use Case ID	UC-05
Use Case Name	Delete Data
Actor(s)	User
Description	Allows the user to permanently delete an existing password entry from the vault after confirming the action.
Pre-conditions	The user has selected a password entry from Manage Password (UC-02) and choose option “delete”.
Post-conditions	The selected password entry is permanently removed from the vault.
Main Flow	1. The system ask user for authentication and validates it. 2. The system retrieves and decrypts the selected password entry 3. The system prompts the user to confirm the deletion. 4. The user confirms it. 5. The system removes the password entry from the vault. 6. The system displays a confirmation message. 7.
Alternate Flow	• If the authentication is not valid, the system will ask the user to input again until it's valid or the user clicks cancel. • The user cancels authentication, the system returns to Manage Password (UC-02) without changes. • The user cancels the deletion, the system returns to the Manage Password (UC-02) view without changes.
Inherited by	UC-02 (Manage Password)
Priority	Medium

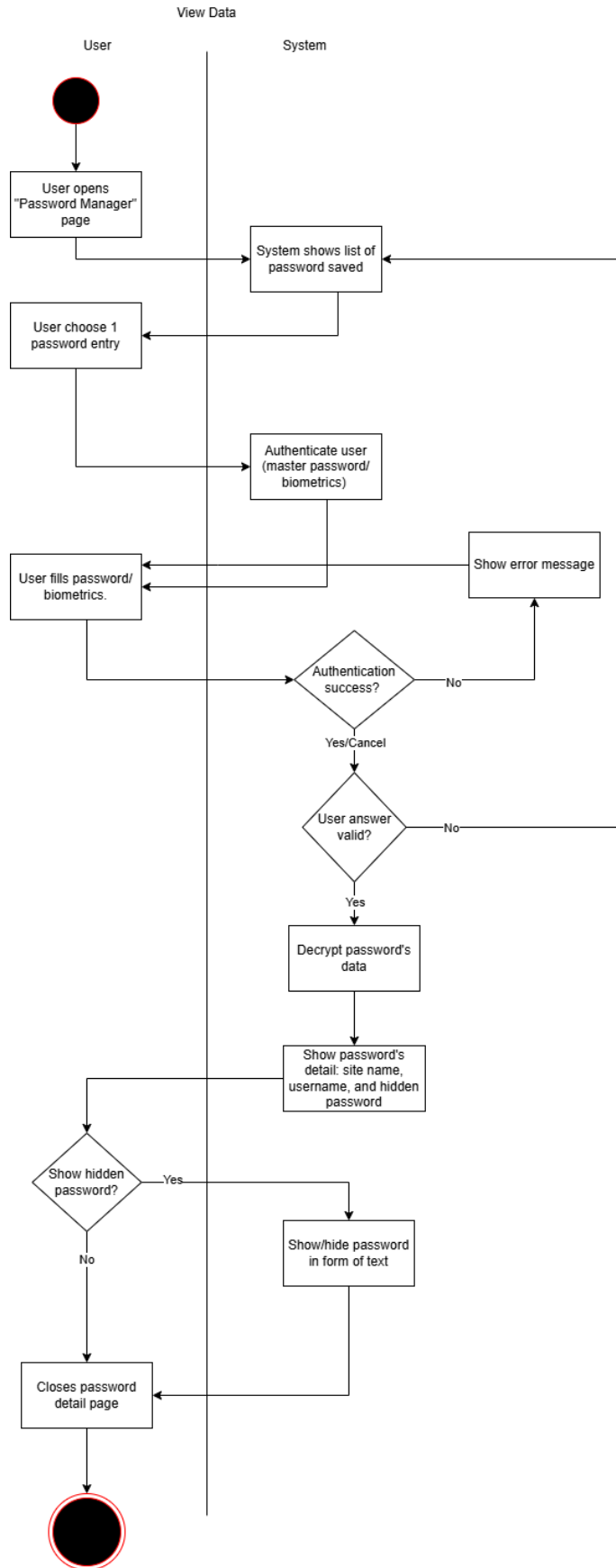
UC-06: Auto-fill Password

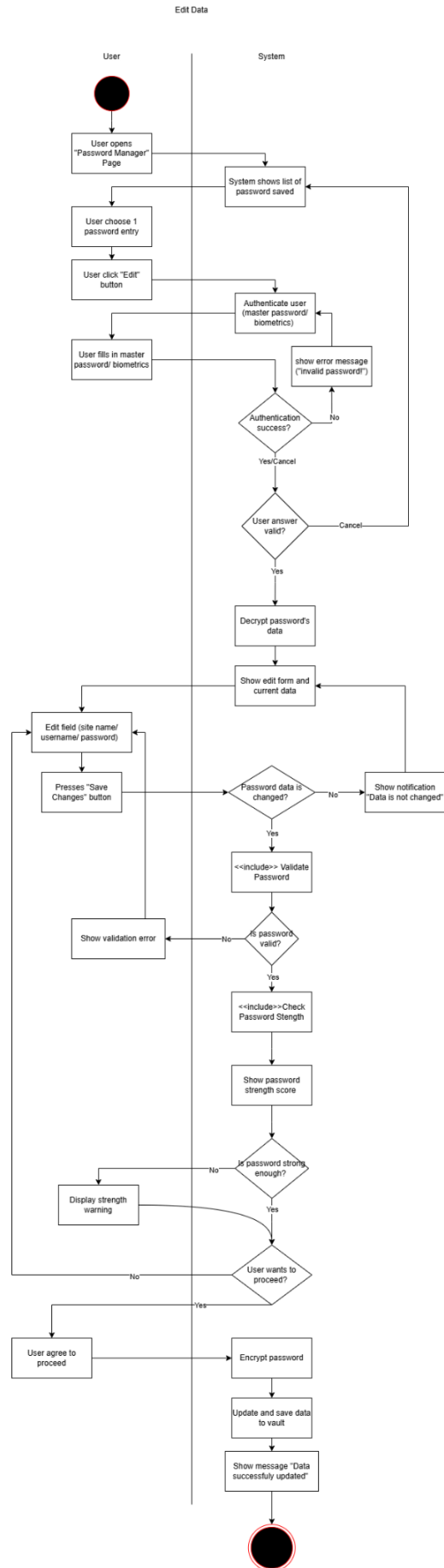
Use Case ID	UC-06
Use Case Name	Auto-fill Password
Actor(s)	User
Description	Allows the user to automatically fill all the login fields on a website or application with stored credentials, eliminating the need to manually look up and copy passwords.
Pre-conditions	The user is logged into SafeVault.
Post-conditions	The login credentials are securely filled into the target fields.
Main Flow	1. The user navigates to a login page. 2. The system detects the URL and matches it against stored entries. 3. The system shows all availed account username. 4. The user chooses an account to use. 5. The system asks for authentication and validates it. 6. The system decrypts the matching password and fills the login fields. 7. The system prompts a success notification.
Alternate Flow	• If there's only one account, the system will straightly ask for authentication. • If the authentication is not valid, the system will ask the user to input again until it's valid or the user clicks cancel. • The user cancels authentication, the system will abort the auto-fill session.
Priority	High

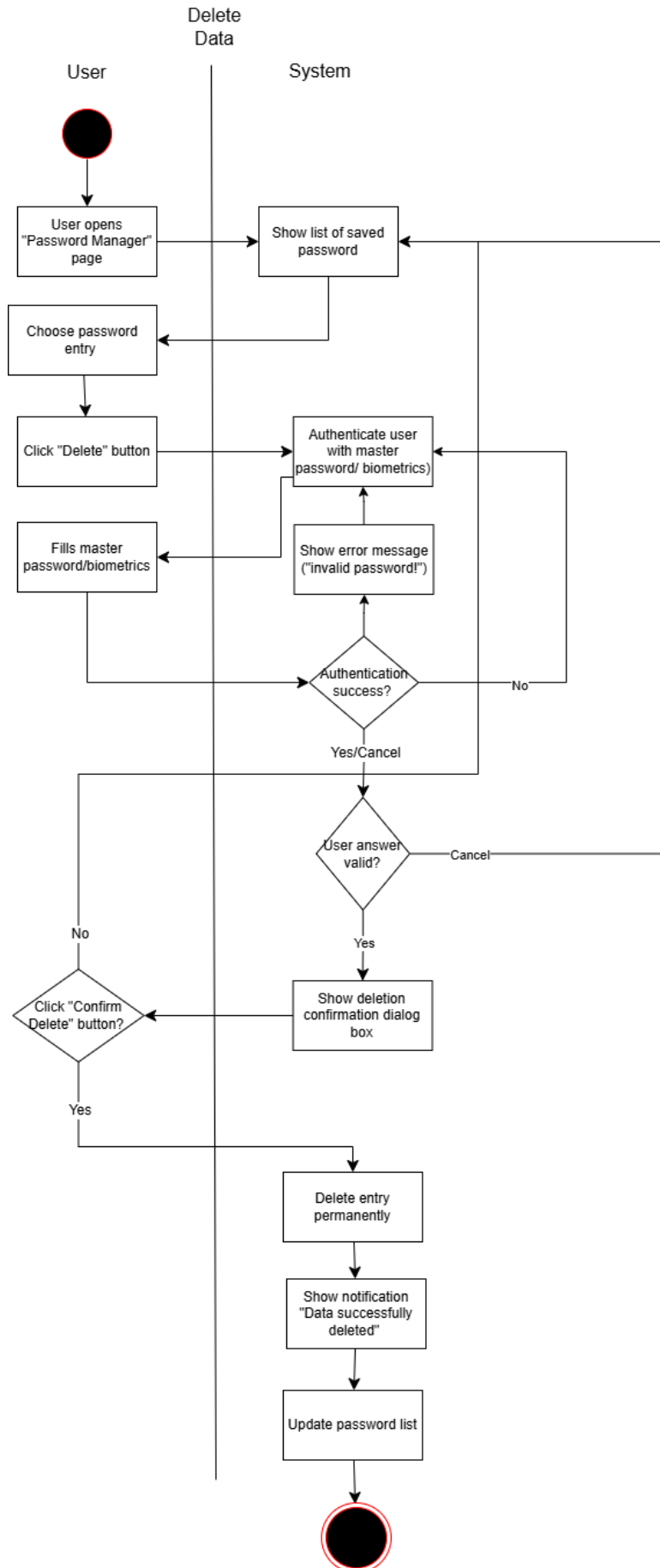
2.3.3 Activity Diagram

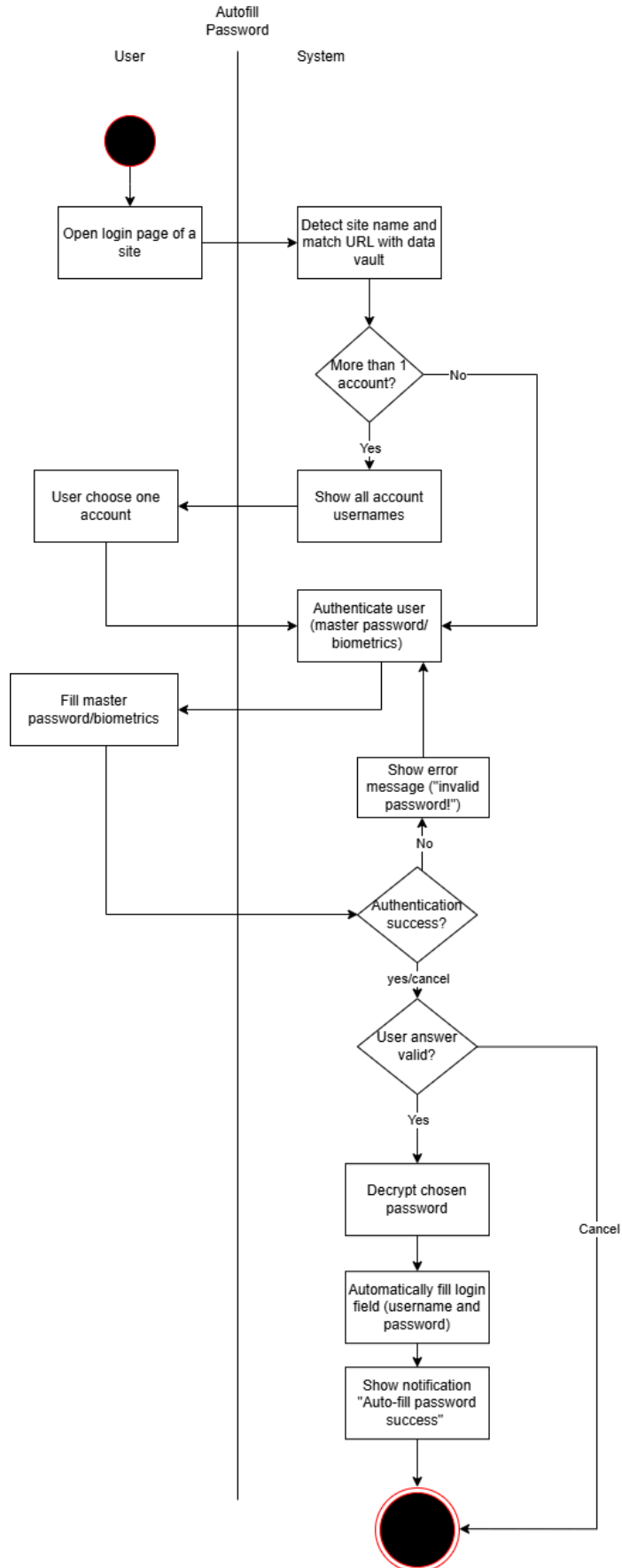




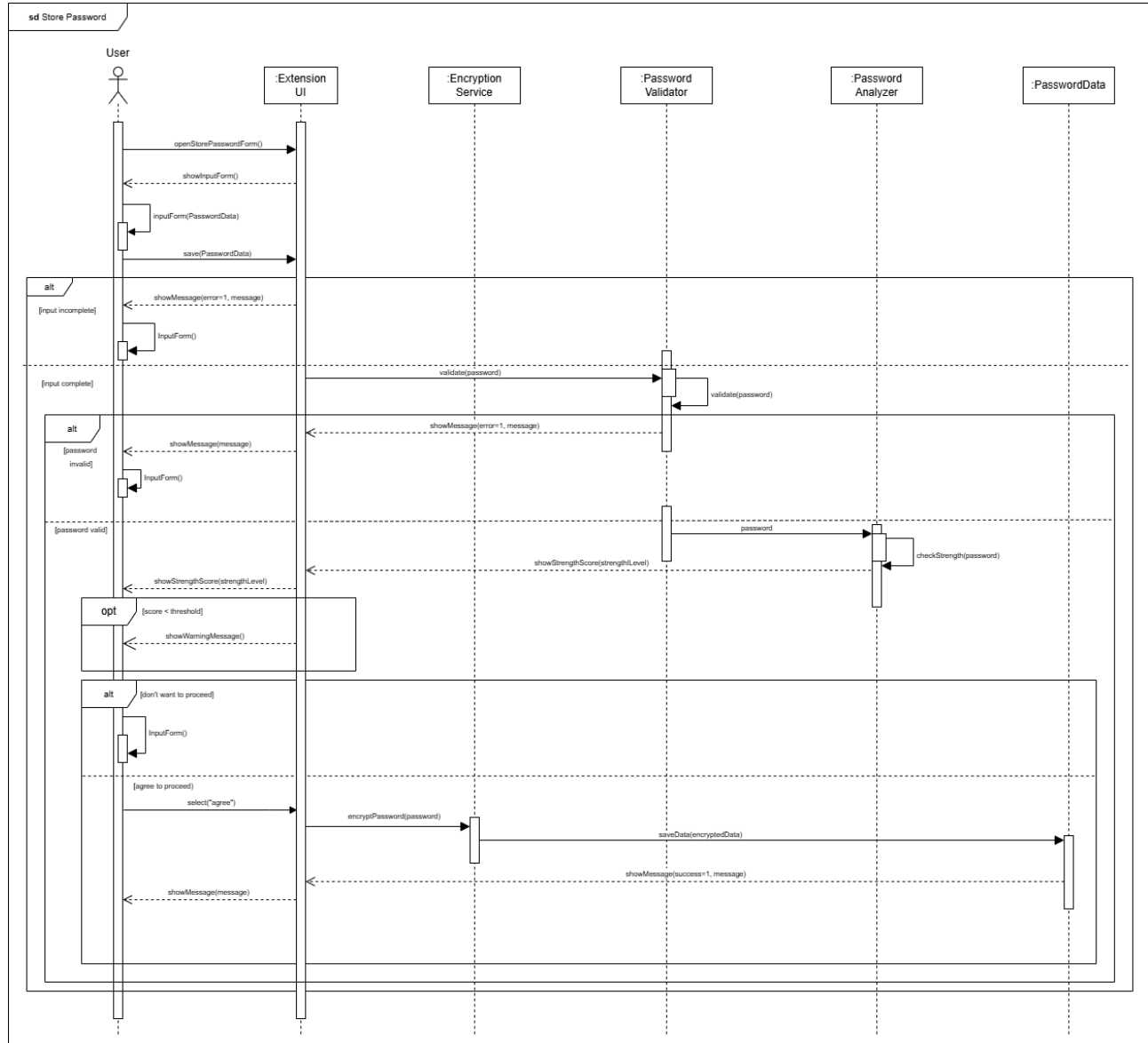


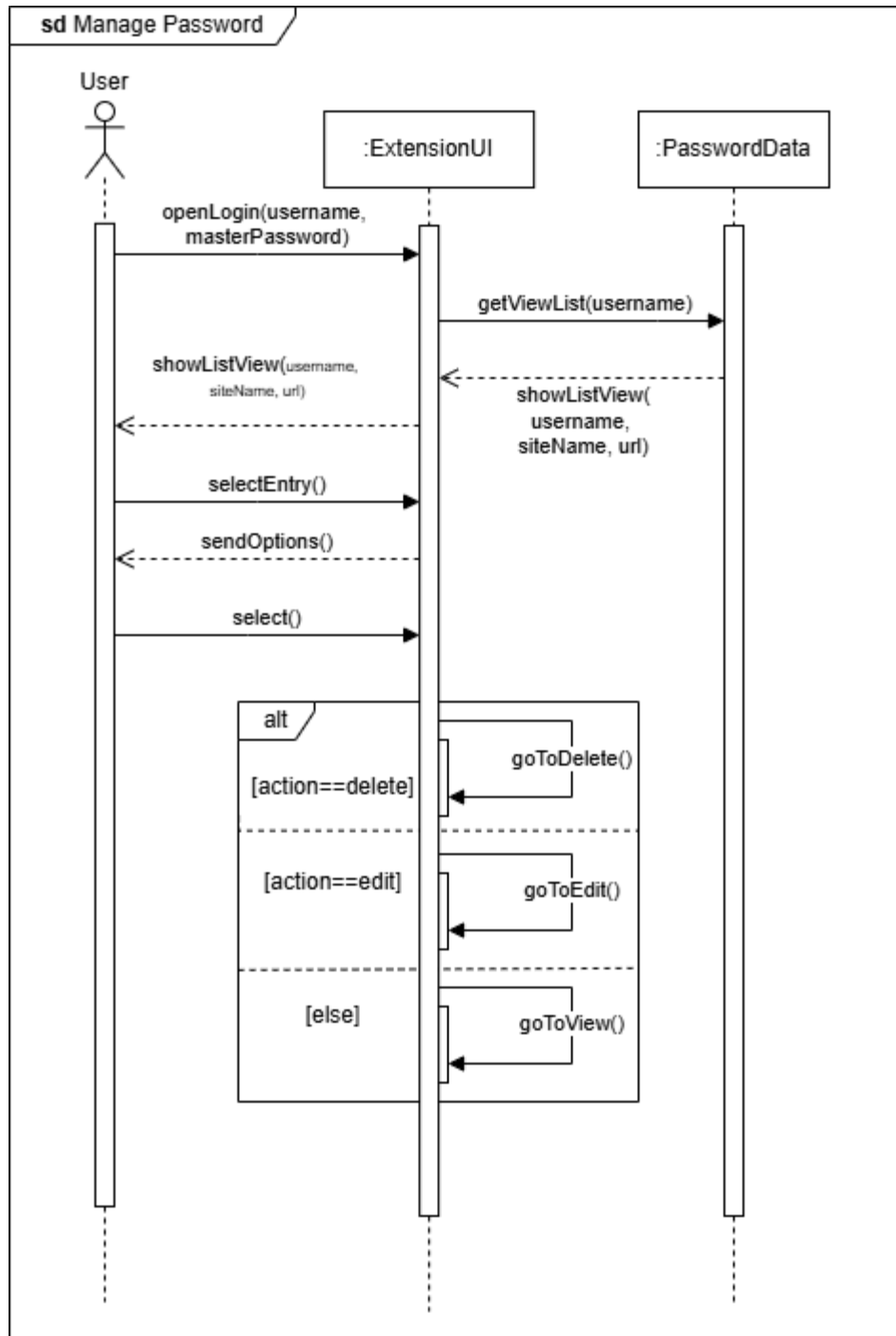


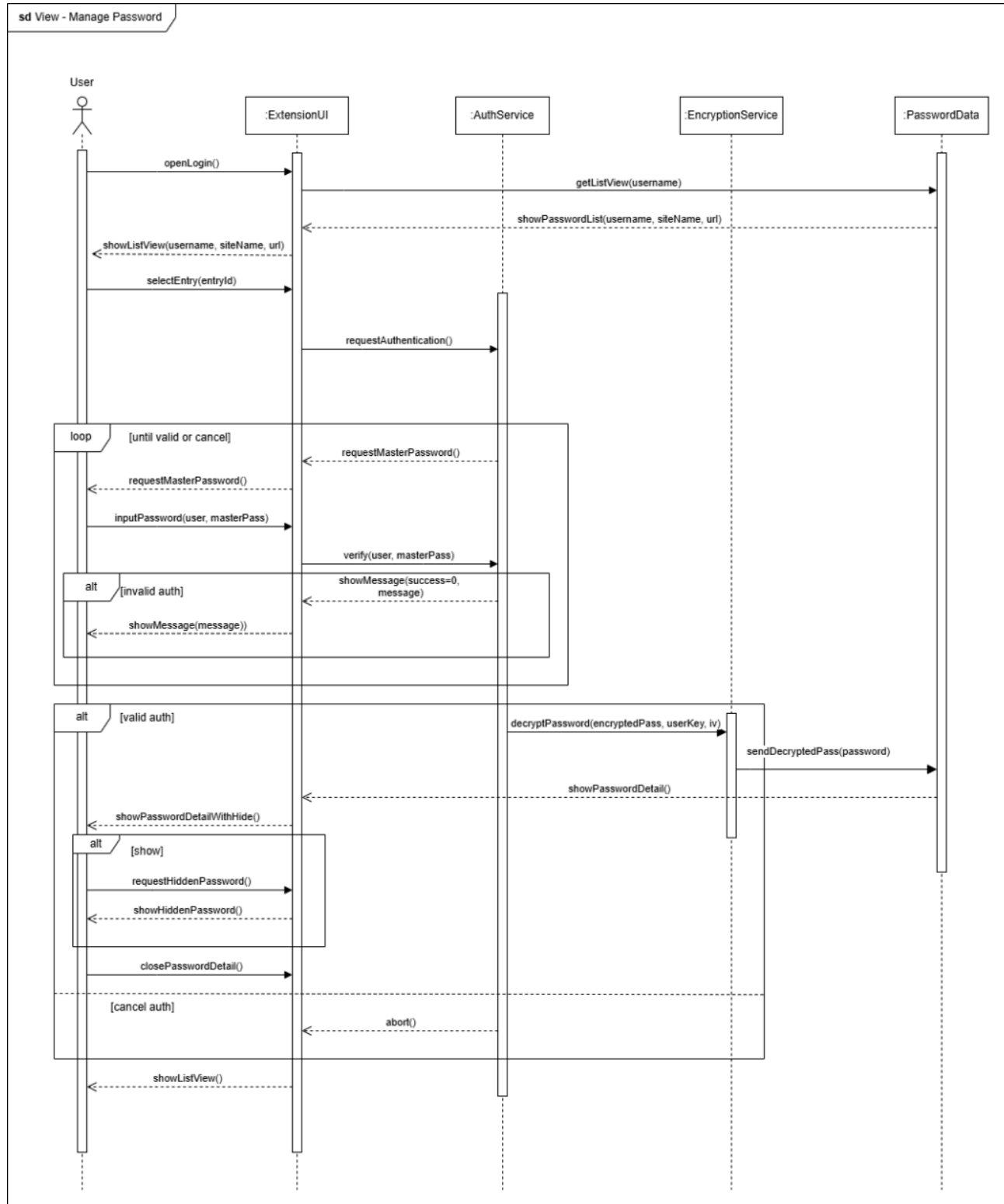


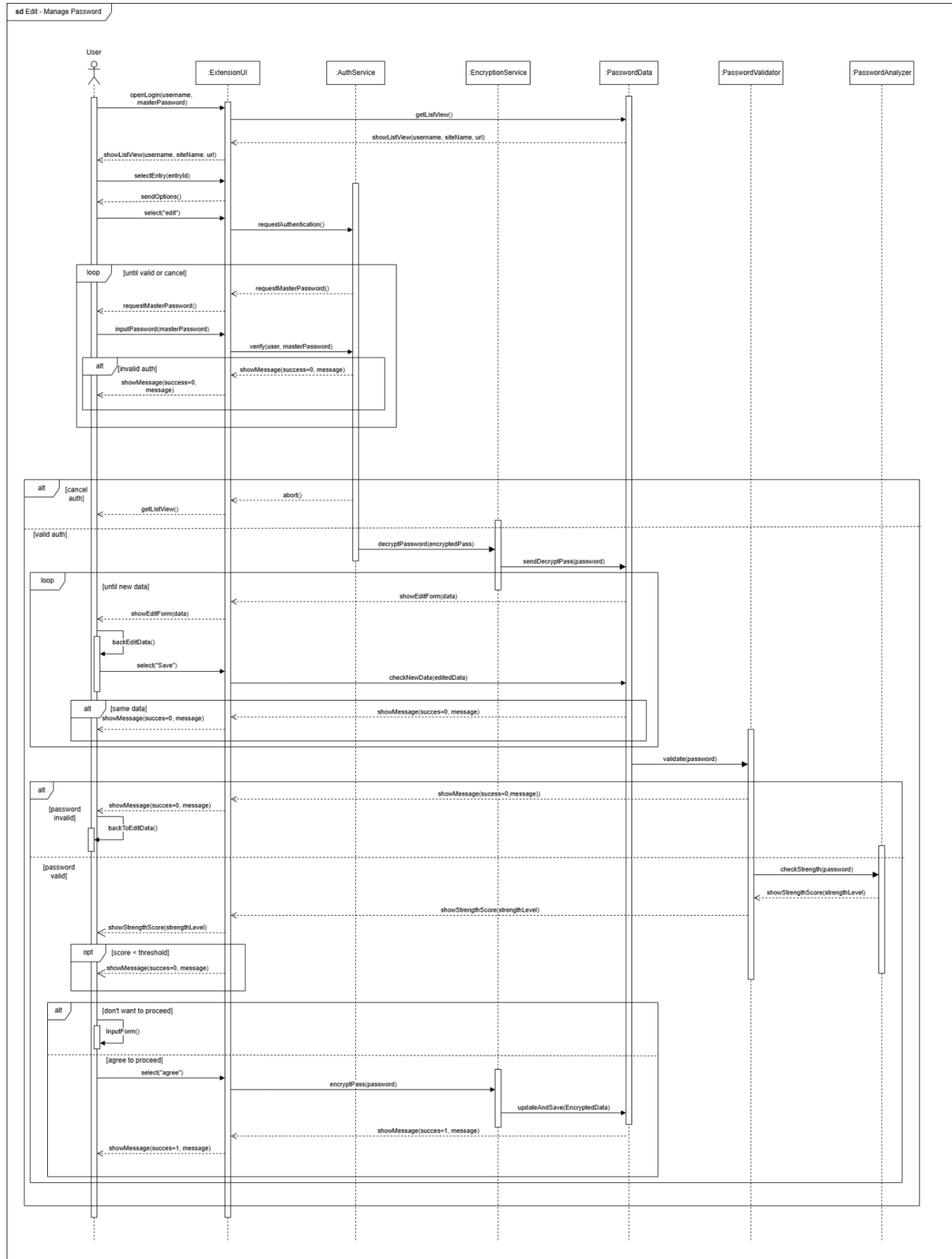


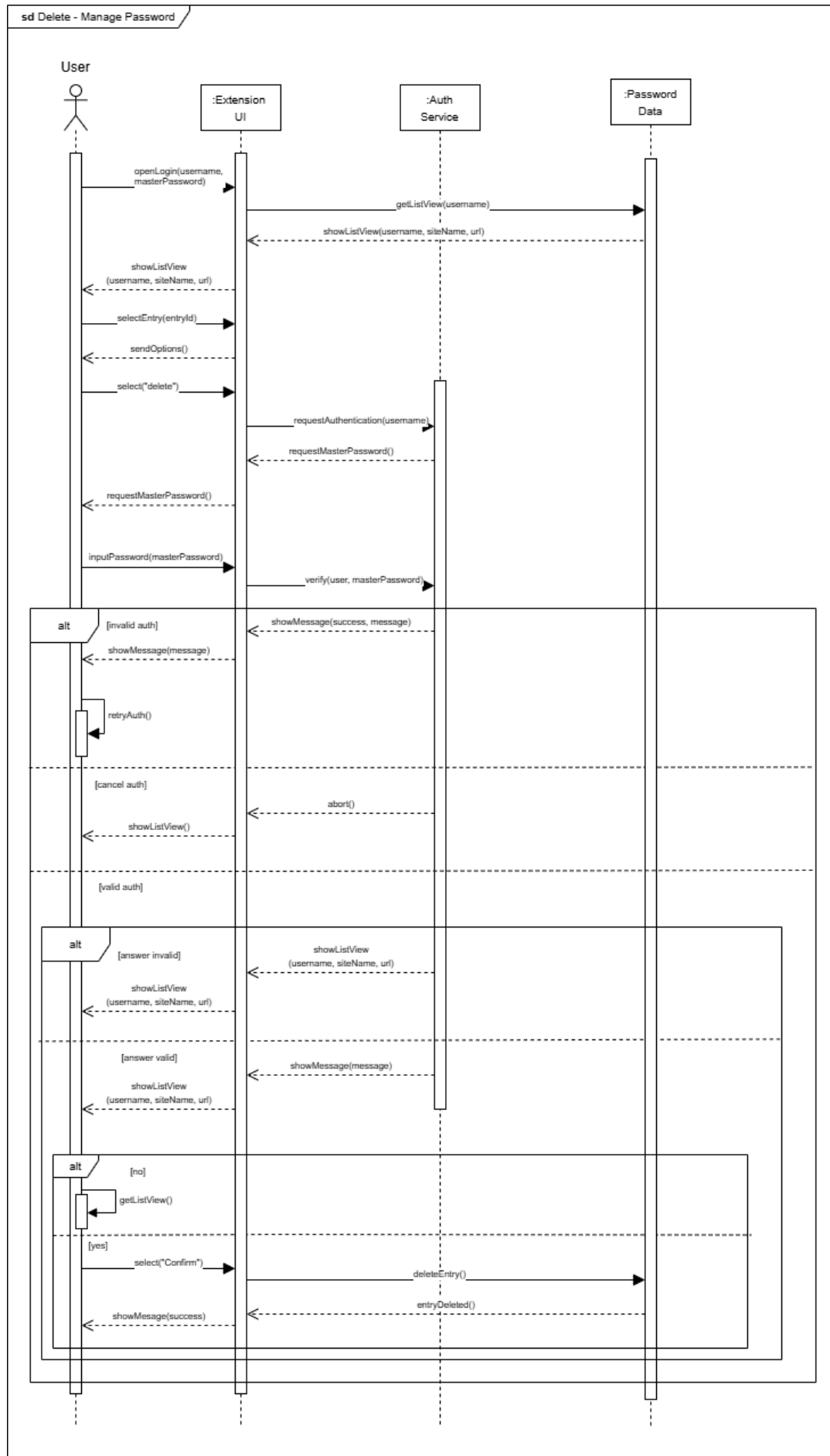
2.3.4 Sequence Diagram

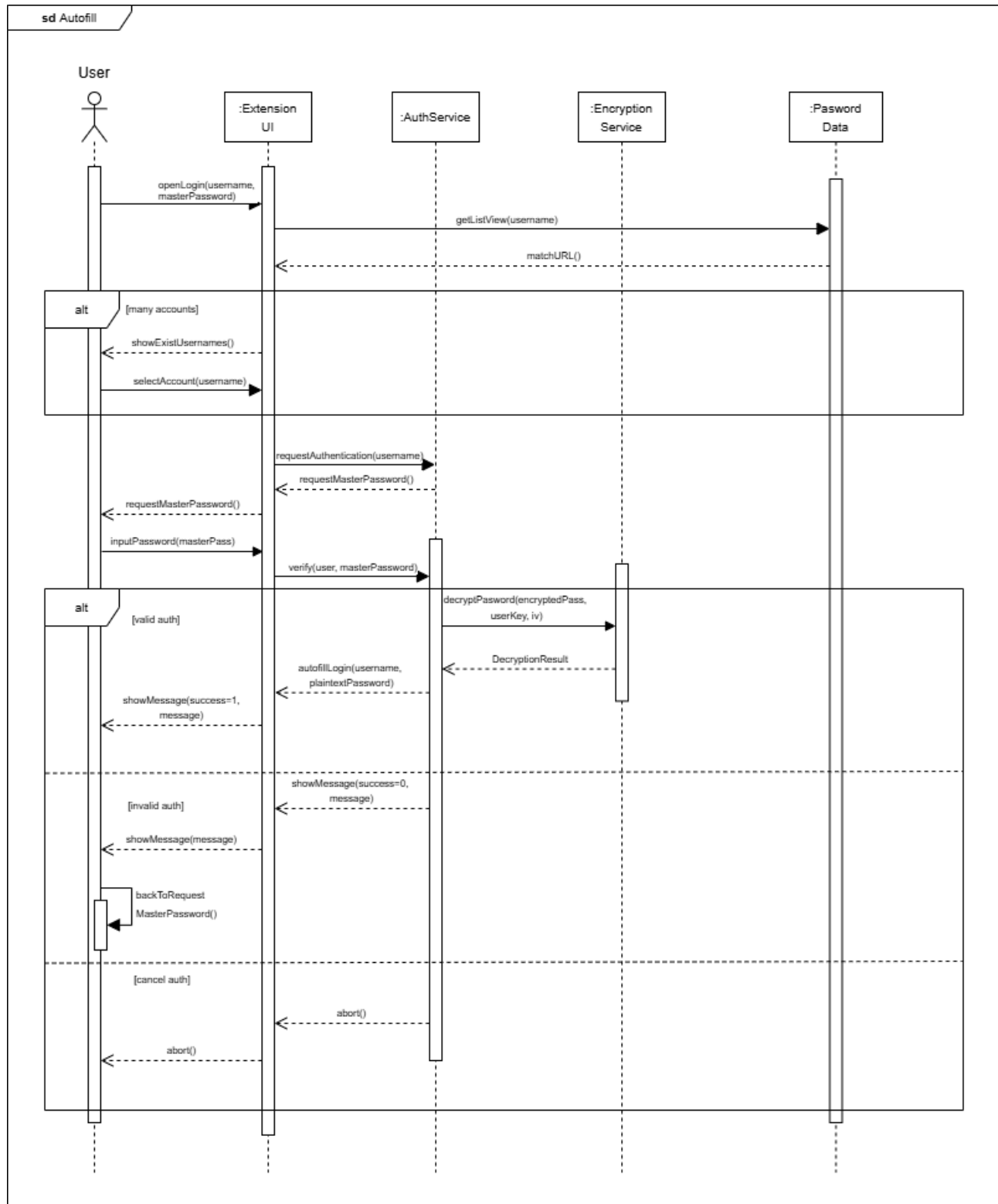




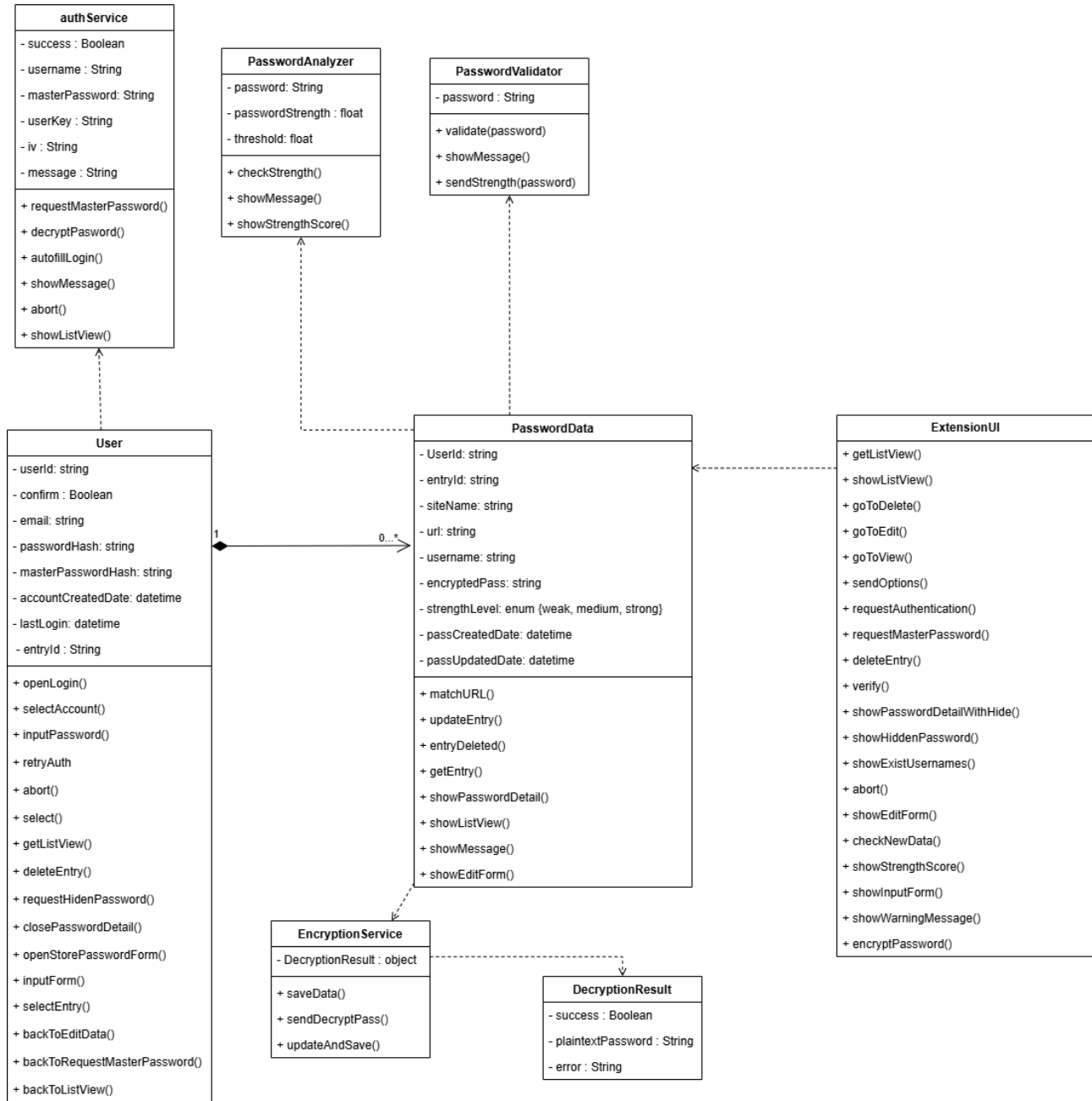








2.3.5 Class Diagram



Berikut ini link detail UML:

https://drive.google.com/drive/folders/18oHeBcDe_v6S9erF1sNFCIFx7ZL1W48T

2.4 Operating Environment

- Hardware: Perangkat komputer (PC/laptop) dengan spesifikasi standar
- Sistem operasi: Windows, macOS, Linux
- Platform: Web-based (SaaS).
- Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, Safari
- Konektivitas: Membutuhkan koneksi internet untuk proses sinkronisasi data ke penyimpanan cloud
- Database & Backend: Google Firebase, menggunakan layanan Firestore sebagai database NoSQL cloud dan Firebase Auth untuk sistem autentikasi pengguna.

2.5 Design and Implementation Constraints

- Koneksi Internet: Fungsi sinkronisasi antar perangkat sepenuhnya bergantung pada ketersediaan jaringan internet. Jika koneksi terputus, pengguna tetap dapat melihat data yang tersimpan di *cache* lokal, namun tidak dapat melakukan pembaruan data ke server cloud.
- Protokol Keamanan: Seluruh komunikasi data wajib menggunakan protokol HTTPS/TLS 1.3. Data kata sandi harus dienkripsi di sisi klien (*client side*) menggunakan algoritma AES-256 sebelum dikirim ke Firebase, sehingga pihak penyedia layanan (Google) tidak dapat membaca data mentah pengguna.
- Kapasitas Server: Meskipun menggunakan Firebase yang bersifat *scalable*, implementasi kode harus efisien dalam melakukan *query* data untuk meminimalkan biaya operasional dan menjaga kecepatan sinkronisasi seiring bertambahnya jumlah kredensial/akun yang disimpan.
- Spesifikasi Perangkat (Minimum):
 - Processor: Minimal Dual-Core 1.6 GHz (untuk menangani proses enkripsi/dekripsi JavaScript dengan lancar).
 - Memori (RAM): Minimal 2 GB RAM (disarankan 4 GB agar browser tetap stabil saat membuka banyak tab).
 - Penyimpanan: Ruang kosong minimal 500 MB untuk *cache* browser dan data lokal aplikasi.
 - Batasan Teknologi: Aplikasi harus dikembangkan menggunakan standar web modern (HTML5, CSS3, ES6+) supaya bisa kompatibel dengan fitur keamanan terbaru pada browser desktop.

2.6 User Documentation

- Online Help & Tutorials: Panduan di dalam aplikasi mengenai cara sinkronisasi antar perangkat.
- Kebijakan Privasi: Dokumen yang menjelaskan bagaimana data dienkripsi sebelum diunggah ke cloud.

2.7 Assumptions and Dependencies

- Asumsi: Pengguna dapat mengakses internet yang stabil untuk memperbaharui data antar perangkat
- Ketergantungan: Sistem bergantung pada keandalan penyedia layanan *cloud* untuk menjamin data selalu tersedia.

3. External Interface Requirements

3.1 User Interfaces

Interface pengguna SafeVault dirancang dengan prinsip *Responsive Web Design* agar dapat digunakan dengan nyaman di layar desktop.

- Karakteristik Logis:
 - Side Navigation Bar: Menggunakan panel navigasi di sisi kiri (*sidebar*) untuk akses cepat ke kategori (seperti: *All Items*, *Security*, *Settings*).
 - Main Content Area: Area tengah yang luas untuk menampilkan daftar kredensial/akun dalam bentuk *list* (*list view*).
 - Detail Panel: Saat salah satu akun diklik, detail informasinya akan muncul di panel sebelah kanan tanpa berpindah halaman (*split view*).
- Standar GUI: Mengikuti prinsip *Material Design* untuk memberikan kesan modern dan bersih. Menggunakan skema warna yang memberikan rasa aman (misalnya: biru tua (*navy*) sebagai warna utama pada *interface* SafeVault).
- Layout Constraints:
 - Fixed Sidebar: Navigasi samping tetap terlihat supaya pengguna dapat berpindah antar menu dengan mudah.
 - Bottom Action Bar: Tombol "Add Entry" diletakkan secara statis pada bagian bawah *kiri* di dalam *side bar* agar pengguna selalu melihatnya tanpa discroll layarnya.
 - Search Bar: Akan diletakkan di posisi tengah atas supaya dapat diakses dengan cepat.

Berikut ini link Figma (prototype):

<https://www.figma.com/proto/9G7zb8yIyxIB4V30wmrJTA/SE---SafeVault?node-id=0-1&t=r6NK4nLyb9THntNT-1>

3.2 Hardware Interfaces

Karena SafeVault beroperasi sebagai *web browser* di komputer, interaksi perangkat kerasnya meliputi:

- Perangkat Input Utama: *Keyboard* dan *mouse/trackpad* supaya pengguna bisa melakukan navigasi dan pengisian formulir.
- Display: Monitor dengan resolusi minimal 1024x768 piksel untuk memastikan seluruh panel (*sidebar*, *main area*, dan *detail panel*) dapat terlihat dengan jelas.
- Network Interface Card (NIC): Memerlukan Wifi yang aktif untuk menghubungkan perangkat ke server cloud agar dapat melakukan sinkronisasi data antar perangkat.
- Penyimpanan Lokal: Memerlukan ruang penyimpanan minimal pada SSD/HDD untuk menyimpan *cache* aplikasi dan file instalasi (jika aplikasi desktop berupa *native*).

3.3 Software Interfaces

SafeVault berinteraksi dengan beberapa lapisan perangkat lunak untuk menjalankan fungsinya, seperti:

- Operating System: Berjalan di atas Windows, macOS, atau Linux.
- Web Browser: Berinteraksi dengan mesin browser (V8 Engine, Gecko, dll.) untuk menjalankan enkripsi.

- Database System (Cloud): Berinteraksi dengan database cloud (seperti Google Firebase atau MongoDB) melalui API untuk menyimpan dan mengambil data kredensial/akun yang sudah terenkripsi.
- Cryptographic Libraries: Menggunakan library seperti *OpenSSL* atau *CryptoJS* untuk melakukan proses enkripsi AES-256 secara lokal di komputer sebelum data dikirim ke internet.
- Local Storage API: Menggunakan penyimpanan lokal browser atau sistem file OS untuk menyimpan sesi login sementara agar pengguna tidak perlu login berulang kali selama sesinya aktif.

3.4 Communications Interfaces

Sebagai sistem yang mendukung sinkronisasi lintas perangkat, standar komunikasi yang digunakan adalah:

- Protokol Jaringan: Menggunakan HTTPS (TLS 1.3) untuk menjamin keamanan jalur komunikasi antara komputer desktop dan server database cloud.
- Data Interchange Format: Seluruh pertukaran data (seperti pengiriman password terenkripsi) menggunakan format JSON.
- Synchronization Mechanism: Menggunakan metode *Long Polling* atau *WebSockets* yang memungkinkan desktop menerima pembaruan secara instan jika ada perubahan data yang dilakukan melalui perangkat lain.
- Network Security: Implementasi *Firewall friendly* (biasanya melalui port 443) untuk memastikan web tetap bisa melakukan sinkronisasi meskipun berada di balik jaringan kantor atau publik yang diproteksi.

4. System Features

<This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product. You may prefer to organize this section by use case, mode of operation, user class, object class, functional hierarchy, or combinations of these, whatever makes the most logical sense for your product.>

4.1 System Feature 1

<Don't really say "System Feature 1." State the feature name in just a few words.>

4.1.1 Description and Priority

<Provide a short description of the feature and indicate whether it is of High, Medium, or Low priority. You could also include specific priority component ratings, such as benefit, penalty, cost, and risk (each rated on a relative scale from a low of 1 to a high of 9).>

4.1.2 Stimulus/Response Sequences

<List the sequences of user actions and system responses that stimulate the behavior defined for this feature. These will correspond to the dialog elements associated with use cases.>

4.1.3 Functional Requirements

<Itemize the detailed functional requirements associated with this feature. These are the software capabilities that must be present in order for the user to carry out the services provided by the feature, or to execute the use case. Include how the product should respond to anticipated error conditions or invalid inputs. Requirements should be concise, complete, unambiguous,

verifiable, and necessary. Use “TBD” as a placeholder to indicate when necessary information is not yet available.>

<Each requirement should be uniquely identified with a sequence number or a meaningful tag of some kind.>

REQ-1:

REQ-2:

4.2 System Feature 2 (and so on)

5. Other Nonfunctional Requirements

5.1 Performance Requirements

<If there are performance requirements for the product under various circumstances, state them here and explain their rationale, to help the developers understand the intent and make suitable design choices. Specify the timing relationships for real time systems. Make such requirements as specific as possible. You may need to state performance requirements for individual functional requirements or features.>

5.2 Safety Requirements

<Specify those requirements that are concerned with possible loss, damage, or harm that could result from the use of the product. Define any safeguards or actions that must be taken, as well as actions that must be prevented. Refer to any external policies or regulations that state safety issues that affect the product’s design or use. Define any safety certifications that must be satisfied.>

5.3 Security Requirements

<Specify any requirements regarding security or privacy issues surrounding use of the product or protection of the data used or created by the product. Define any user identity authentication requirements. Refer to any external policies or regulations containing security issues that affect the product. Define any security or privacy certifications that must be satisfied.>

5.4 Software Quality Attributes

<Specify any additional quality characteristics for the product that will be important to either the customers or the developers. Some to consider are: adaptability, availability, correctness, flexibility, interoperability, maintainability, portability, reliability, reusability, robustness, testability, and usability. Write these to be specific, quantitative, and verifiable when possible. At the least, clarify the relative preferences for various attributes, such as ease of use over ease of learning.>

5.5 Business Rules

<List any operating principles about the product, such as which individuals or roles can perform which functions under specific circumstances. These are not functional requirements in themselves, but they may imply certain functional requirements to enforce the rules.>

6. Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

Appendix B: Analysis Models

<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>

Appendix C: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>